SYSTEM ARCHITECTURE · TEAM PROJECT

EcomApp

A buyer-and-seller marketplace, delivered through a collaborative path from application features toward automated deployment.

> **Team:** Salenga · Riboroso · Tato · Ash
> **Roster check:** fifth required member not yet confirmed; add the real member before submission.

Presentation draft · feature-stage checkout

01 · THE APPLICATION

One storefront, three service boundaries

Buyer experience

Browse and search products, manage a cart, check out, and review orders.

Seller experience

Manage owned listings, see relevant orders, and update fulfillment status.

Service modules

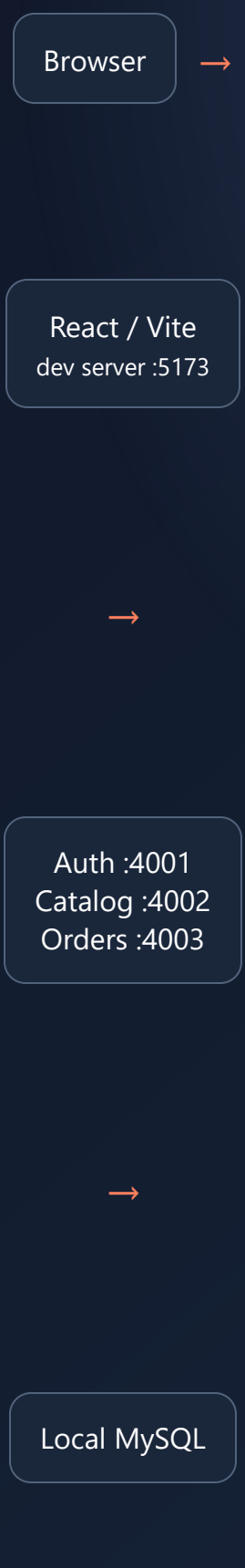
Authentication, catalog, inventory, and order management share MySQL-backed data.

These features are present in the starter baseline. Contributions and final behavior must be attributed to the actual reviewed work, not assumed.

Source: repository README and feature-stage service layout

02 · ARCHITECTURE TODAY

Verified checkout: local feature-stage topology



This is the starter's local-development path. Docker, Compose, deployment NGINX, and Jenkins are not present in this checkout.

Do not present the target deployment as already integrated

03 · TECHNOLOGY CHOICES

Keep the stack aligned to the module boundaries

React + Vite

Component-based buyer/seller UI with a fast development server and a build-time static bundle.

Node.js + Express

Three independently organized APIs make authentication, catalog, and order ownership visible.

MySQL

Relational tables express users, products, orders, and item relationships.

Deployment tools

Docker, Compose, NGINX, and Jenkins are planned team-stage additions; include their rationale after review.

04 · USER JOURNEYS

Two roles, different actions

Buyer path

Sign up / log in → search → cart → checkout → order history → cancel eligible pending order.

Seller path

Sign up / log in → create, edit, delete listing → view seller orders → advance fulfillment.

Fresh disposable buyer and seller sessions completed the local UI journeys. The earlier seller session returned 403 on protected requests; its token issue was not diagnosed. Final deployment-proxy QA is still outstanding.

05 · VISIBLE BUILD IDENTITY

Make the browser tell the truth about the bundle

Build-time source

`VITE_BUILD_ID` is compiled into the UI at build time. An unset value is labeled “unlabeled build”.

Release rule

CI should set it to the actual checked-out commit or release ID before building the frontend image. Never ship a hand-typed build claim.

The label is displayed on both the sign-in screen and the signed-in header.

Sign-in · signed-in local QA screenshots · `local-qa-20261013` (not a deployment claim).

Verify the merged image and visible value together in the final proxy

06 · DOCKER

Containerization is a team integration gate

Planned deliverable

Reviewed custom Dockerfiles and ignore rules for team-authored application modules, using reproducible builds.

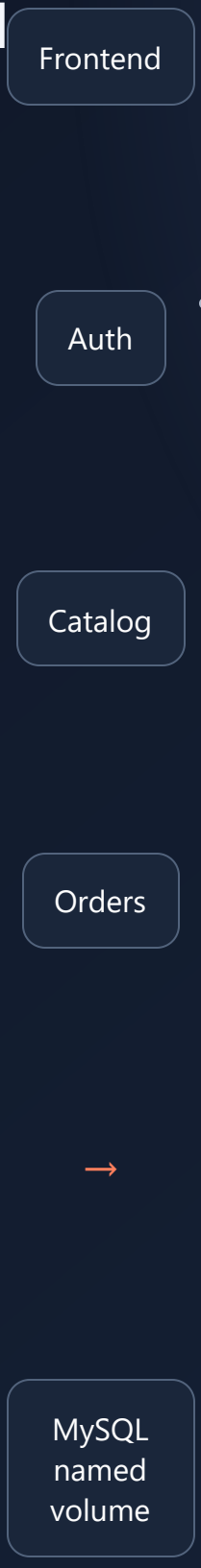
Evidence required

Successful image builds, image/source tags, runtime health, and a clean configuration check.

No Dockerfiles or Compose configuration are present in this feature-stage checkout. This slide is a plan, not a completion claim.

07 · COMPOSE & PERSISTENCE

One-command startup needs durable state



- Agree health checks, service dependencies, private networks, and environment settings with infrastructure/backend owners.
- Prove persistence by writing safe disposable data, restarting the stack, and reading it back.

Compose, a named database volume, and persistence proof are not in this checkout.

08 · JENKINS & TRIGGER

Push → checks → tested build → deploy

Git push

→

Jenkins trigger

→

Quality gate

→

Deploy & verify

Final demo evidence should include an automatic trigger without “Build Now”, a successful pipeline, and a deliberately failing test that blocks deployment.

No Jenkinsfile, Jenkins stack, or automated trigger is present in this checkout. Coordinate implementation and run IDs with Riboroso.

09 · TESTS & QUALITY

Quality checks should fail loudly and early

Available starter checks

`npm run lint`

`npm run build:frontend`

`npm run check:services`

Local integration smoke

`npm run test:smoke` creates unique buyer/seller/product records, asserts search, checkout, seller fulfillment and buyer history, cleans up, and exits nonzero on failure. Backend-owner review and CI wiring remain.

Record actual command output and pipeline status in the final deck after running the checks on the submission revision.

10 · PERSISTENCE PROOF

Show the data survives a restart

1. Record a disposable test record and the active stack/image identifier.
2. Restart the application containers without deleting the named DB volume.
3. Read the record back through the application path and capture redacted evidence.

This is a demo transition, not existing evidence: no container stack or volume proof is available in the current checkout.

11 · ROLLBACK

Demonstrate recovery, not just a rollback button

Before

Record the known-good deployed image tag, source commit, and health response.

After

Redeploy that known-good tag, verify the proxy and UI, and record the pipeline run and resulting label.

Rollback behavior must be verified with Riboroso and Tato on the actual deployment host before presentation.

12 · QA SNAPSHOT

Local journeys passed; proxy still needs proof

15 / 15

Checklist scenarios passed locally

Disposable buyer and seller UI journeys and buyer role-boundary requests passed against local services.

403

Role boundary confirmed

Buyer requests to seller-only APIs return 403 as expected. An earlier session also showed 403; its token cause is unknown.

See docs/qa-checklist.md and the dated execution record. Repeat all flows through the final proxy.

13 · OWNERSHIP & NEXT STEPS

Coordinate the handoffs before the final demo

Ash · Frontend / QA / docs

Build label, manual QA, README, smoke-test collaboration, technical outline, and this draft deck.

Riboroso · Backend / CI-CD

API/test explanation and smoke-test/pipeline coordination; confirm deliverables and run evidence.

Confirm and assign the fifth distinct member; the course guide explicitly says not to invent a name or double-count a role.

Tato · Infrastructure

Architecture, containers, Compose, proxy, persistence, and deployment evidence.

Salenga · Project lead

Roster, issues, decisions, contributions, integration checkpoints, and presentation flow.

14 · REFERENCES & DEMO TRANSITION

From feature baseline to verified delivery

- FinalProject-CI-CD-Automation · reference project
- This checkout's README · feature-stage local setup and build-label source
- Frontend QA checklist · actual browser observations and unrun journeys
- Technical document outline · evidence needed for the PDF

Demo transition: show the actual proxy route, complete disposable buyer and seller journeys, pipeline run, persisted record, and verified rollback only after those artifacts exist.

Replace this draft's planned items with dated output from the merged submission revision.